

Parallel Online Examination System Project Report

Table of Content :

1. Group Members
2. Objective
3. Introduction
4. Background
 - 4.1. Threads and concurrency
 - 4.2. Race Conditions
 - 4.3. Mutex
 - 4.4. Semaphores
 - 4.5. Condition Variables
 - 4.6. Producer-Consumer Pattern
5. Platform and Language
6. Methodology
 - 6.1. System Architecture
 - 6.2. Thread Responsibilities
 - 6.3. Synchronization Design
 - 6.4. Cheating detection
 - 6.5. Time limit Enforcement
 - 6.6. Logging
 - 6.7. Analytics
7. Conclusion

1. GROUP MEMBERS:

- 1.1. 24K3014-Mahad Awan
- 1.2. 24K3044-M.Basim Iftikhar
- 1.3. 24K3049-Huzaifa Masood

2. Objective:

The Objective of this project is to design and implement a Parallel Online Examination System that simulates a real-world Learning Management System (LMS) backend. The system demonstrates core operating system concepts including multithreading, process synchronization, mutual exclusion, and semaphore-based resource management. The project handles multiple students attempting exams simultaneously while evaluator threads grade responses in parallel, with all shared data protected through proper synchronization mechanisms.

3. Introduction

Modern online examination platforms must handle hundreds of concurrent users , students submitting answers, evaluators grading responses, and systems logging activity all at the same time. Without proper synchronization, such systems would generate incorrect results, crash under load, or lose data entirely.

This project implements a scaled-down but architecturally accurate simulation of such a system using the C programming language and POSIX threads from pthreads.h library. The system creates student threads that submit exam answers, evaluator threads that grade submissions, and supporting threads for auto-saving, logging, and live dashboard display.

4. Background

4.1. Threads and Concurrency:

A thread is an independent flow of execution within a process. Unlike processes, threads share the same memory space, which allows efficient communication but introduces the risk of race conditions when multiple threads access shared data simultaneously.

4.2. Race Conditions:

A race condition occurs when two or more threads access shared data concurrently and the final result depends on the timing of their execution. For example, two evaluator threads simultaneously reading and writing to the same result slot would produce corrupted data.

4.3. Mutex (Mutual Exclusion):

A mutex is a synchronization primitive that allows only one thread to access a critical section at a time. Threads attempting to lock an already-locked mutex are blocked until it is released. In this project, mutexes protect the results array, submission queue, and log file.

4.4. Semaphores:

A semaphore is a counter-based synchronization primitive. Unlike a mutex which allows only one thread, a semaphore with value N allows up to N threads to enter a section simultaneously. This project uses a semaphore to limit the number of concurrent evaluators actively grading at any moment.

4.5. Condition Variables:

Condition variables allow threads to sleep and wait for a specific condition to become true, rather than spinning and wasting CPU cycles. Evaluator threads use condition variables to sleep when the submission queue is empty and wake up when a new submission arrives.

4.6. Producer-Consumer Pattern:

This project implements the classic producer-consumer pattern where student threads act as producers adding submissions to a queue, and evaluator threads act as consumers picking from that queue and grading them.

5. Platform and Languages

5.1. Language: C (C99 standard)

5.2. Operating System: Linux (Ubuntu)

5.3. Libraries Used:

5.3.1. `pthread.h` — POSIX threads, mutexes, condition variables

5.3.2. `semaphore.h` — POSIX semaphores

5.3.3. `time.h` — timestamps and time measurement

5.3.4. `fcntl.h` — file operations (open, write, close)

5.3.5. `unistd.h` — sleep function

5.3.6. `string.h` — string operations

5.4. Compilation Command:

5.4.1. `gcc -o system exam_system.c -lpthread`

6. Methodology

6.1. System Architecture

The system consists of the following concurrent components running simultaneously:

1. Student Threads (1000)
 - submit to
2. Submission Queue (mutex protected)
 - ↓ consumed by
3. Evaluator Threads (15)
 - ↓ write to
4. Results Array (mutex protected)
 - ↓ read by
5. Dashboard Thread + AutoSave Thread + Analytics

6.2. Thread Types and Responsibilities

Student Thread (*std_thread*): Each student is represented by an independent thread. The thread simulates exam-taking time using `sleep(rand() % MAXSTUDENTEXAMTIME)`. After the sleep, the thread checks whether it exceeded the time limit. Three scenarios are possible:

1. Honest submission: student finished within time limit, submits own score
2. Timeout: student exceeded time limit and chose not to cheat, score = 0
3. Cheating: student exceeded time limit, randomly decides to copy another student's score from the submission queue

Evaluator Thread (*eva_thread*): Evaluator threads continuously consume submissions from the queue using the producer-consumer pattern with condition variables. Each evaluator:

1. Waits on condition variable when queue is empty
2. Wakes up when signaled by a student submission
3. Picks a submission, grades it
4. Stores result in the protected results array
5. A semaphore limits concurrent grading to `MAXEVALUATORLIMIT` at a time

AutoSave Thread (autoSave): Runs independently every AUTOSAVEROUTINE seconds. Opens autoSave.txt with O_TRUNC, writes current results snapshot, and closes the file. This ensures partial results are preserved even if the system crashes mid-exam.

Dashboard Thread (dashboard): Clears the terminal and reprints live exam statistics every second including students submitted, graded, timed out, cheaters caught, currently grading, and time elapsed.

6.3. Synchronization Design

Shared Resource	Protection Mechanism
submissionQueue[]	submissionLock (mutex)
studentResults[]	resultLock (mutex)
examOver -flag	examLock (mutex)
Log file (examRecords.txt)	logLock (mutex)
Concurrent graders	gradingLimit (semaphore)
Queue empty wait	submissionCondition (condition variable)

6.4. Cheating Detection

Students who exceed the exam time limit are assigned a random cheatFlag (0 or 1). If cheatFlag is 1 and submissions exist in the queue, the student copies a randomly selected submitted student's score. The cheated flag and source student ID are stored with the submission. Evaluators detect and penalize cheating by setting the final score to 0.

6.5. Time Limit Enforcement

Each student thread records its start time using time(NULL) before sleeping. After waking, it calculates elapsed time using difftime(). If elapsed time exceeds EXAMTIMELIMIT (3 seconds), the student is considered over time.

6.6. Logging

Every significant event is written to examRecords.txt with a formatted timestamp using strftime(). The log file uses O_APPEND mode to preserve history across runs. All writes are protected by logLock mutex to prevent interleaved log entries.

6.7. Analytics

After all threads complete, generateAnalytics() processes studentResults[] without any mutex (since all threads have finished) and computes pass count, fail count, cheat count, timeout count, average score of honest students, highest score, and lowest score. Results are written to ExamAnalytics.txt.

7. Results

The system was tested with the following configuration:

MAXSTUDENTS = 100
MAXEVALUATOR = 20
MAXEVALUATORLIMIT = 5
EXAMTIMELIMIT = 3 seconds
MAXSTUDENTEXAMTIME = 7 seconds

Expected distribution based on probability:

1. Students finishing within 3 seconds (0,1,2,3): 59% HONEST.
2. Students exceeding time limit: 42%:
 - 2.1. Of those, ~50% timeout.
 - 2.2. Of those, ~50% cheat.

The live dashboard confirmed real-time updates and the semaphore correctly limited concurrent graders. AutoSave files were observed updating every 2 seconds with growing result counts.

8. Conclusion

This project successfully demonstrates the core principles of concurrent programming and operating system synchronization in a realistic scenario. The system correctly handles 1000 simultaneous student threads using mutexes to prevent race conditions, semaphores to limit resource usage, and condition variables to eliminate busy-waiting. The producer-consumer architecture cleanly separates student submission from evaluator grading. Additional features including time limits, cheating detection, auto-save, logging, analytics, and a live dashboard make this a complete simulation of a real LMS backend.

